Evaluation of Applying Federated Learning to Distributed Intrusion Detection Systems Through Explainable AI

Ayaka Oki, *Member, IEEE*, Yukio Ogawa, *Member, IEEE*, Kaoru Ota, *Member, IEEE*, and Mianxiong Dong, *Member, IEEE*

***Abstract*—We apply federated learning (FL) to a distributed intrusion detection system (IDS), in which we deploy numerous detection servers on the edge of a network. FL can mitigate the impact of decreased training data in each server and exhibit almost the same detection rate as that of the non-distributed IDS for all attack classes. We verify the effect of FL using explainable artificial intelligence (XAI); this effect is demonstrated by the distance between the feature set of each attack class in the distributed IDS and that in the non-distributed IDS. The distance increases for independent learning and decreases for FL.**

***Index Terms*—Distributed intrusion detection system, deep learning, federated learning, explainable artificial intelligence.**

I. Introduction

INTERNET of Things (IoT) devices have become ubiquitous, leading to an increase in the risk of being attacked from such devices [1]. This letter aims to deploy an intrusion detection system (IDS) whose detection servers are distributed at the edge of a wide-area network (called a distributed IDS) to prevent such attacks. We refer to such detection servers as edge servers. Although deep learning (DL) is a promising technique for IDSs [2], the attack detection rate may decrease in the distributed IDS due to the decrease in the amount of training data per detection server. Furthermore, DL techniques do not provide an explanation for detection decisions. We then apply federated learning (FL) to the distributed IDS to mitigate the impact of decreased training data, and the purpose of our research is to verify the effect of FL by using explainable artificial intelligence (XAI).

Prior studies have summarized machine learning (ML) techniques that are applied to IDSs [1]. Vinayakumar et al. compared the detection accuracy of binary and multi-class classification models created by using multilayer perceptrons (MLP) [2]. Furthermore, FL has gained attention as a method to enhance the detection accuracy of distributed IDS [3]. For example, an FL-based IDS has been developed to improve communication security for connected and autonomous vehicles [4]. Additionally, a personalized federated transfer learning-based detection framework utilizing edge computing has been proposed for detecting attacks on IoT devices [5]. Following these studies, our study also applies FL-based ML to distributed IDS against IoT devices.

XAI has also been highlighted as an approach to enhance transparency for ML-based IDSs, especially when dealing with black-box IDS models [6]. Wang et al. enumerated the features influencing attack detection according to values of Shapley additive explanations (SHAP) [7] by applying a XAI algorithm to the prediction results of binary and multi-class classifications [8], where a SHAP value is the contribution of a feature to a particular detection. Sarhan et al. evaluated the difference in the attack detection accuracy between two techniques, random forest and MLP, and interpreted the detection results by calculating the SHAP value of each feature [9]. Moreover, Barnard et al. used SHAP values to train an autoencoder to distinguish between previously seen and unseen attacks [10]. However, to the best of our knowledge, there has been no study that explains the detection performance of distributed IDSs by using XAI.

To solve this problem, we simulate a distributed IDS in a computational environment using the UNSW-NB15 dataset [11]. Each edge server in a distributed IDS uses DL to classify the attack classes. Furthermore, we evaluate the impact of input features on attack classification by calculating the SHAP values of the features.

The contributions of our study are summarized below:

1) We simulate independent learning (i.e., distributed on-site learning [12]) and FL (i.e., *horizontal federated leaning* [13]) in a distributed IDS and compare them with a non-distributed IDS. We evaluate recall as the attack detection metric and calculate the distance between the feature set of each attack class in the distributed IDS and that in the non-distributed IDS; this distance indicates the similarity between the two feature sets.
2) In independent learning, local training data limited to each edge server leads to lower recall in the server by 21% at maximum, compared to non-distributed IDS. The detection of attack classes which have many primary features significantly degraded in general. Theses attack classes show large differences in recall among the edge

Received 28 July 2024; revised 27 August 2024; accepted 17 September 2024. Date of publication 23 September 2024; date of current version 25 October 2024. This work was supported in part by the JSPS KAKENHI under Grant JP20H04174, Grant JP21K11861, and Grant JP22K11989; in part by the Leading Initiative for Excellent Young Researchers (LEADER), MEXT, Japan; and in part by the JST, PRESTO, Japan, under Grant JPMJPR21P3. The associate editor coordinating the review of this article and approving it for publication was L. Foschini. *(Corresponding author: Yukio Ogawa.)*

Ayaka Oki is with the School of Information and Electronic Engineering, Muroran Institute of Technology, Muroran 050-8585, Japan (e-mail: ayaka.o.ovo.68@gmail.com).

Yukio Ogawa is with the Faculty of Engineering and Design, Kagawa University, Takamatsu 761-0396, Japan (e-mail: ogawa.yukio@kagawa-u.ac.jp).

Kaoru Ota and Mianxiong Dong are with the Department of Information and Electronic Engineering, Muroran Institute of Technology, Muroran 050-8585, Japan (e-mail: ota@muroran-it.ac.jp; mx.dong@csse.muroran-it.ac.jp).

Digital Object Identifier 10.1109/LNET.2024.3465516

2576-3156 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 07:59:22 UTC from IEEE Xplore. Restrictions apply.

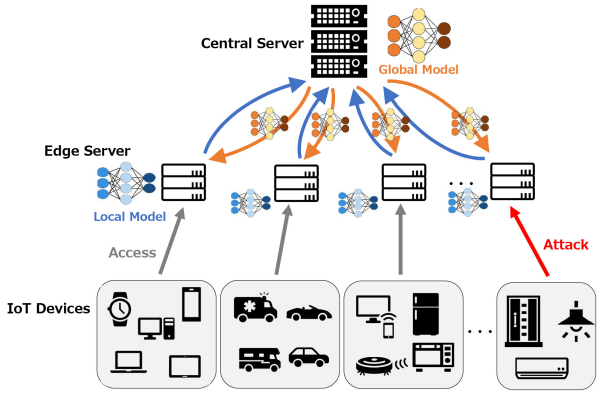


Fig. 1. System model.

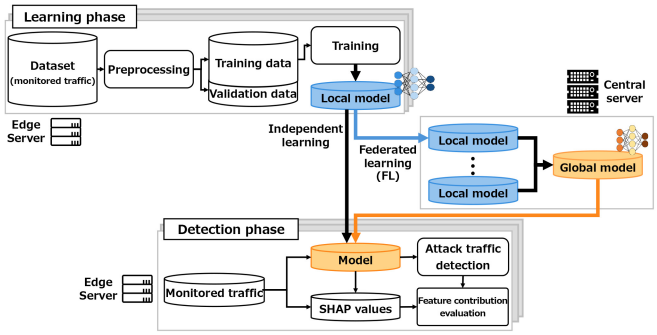


Fig. 2. System process.

servers, as indicated by how the average future distance varies correspondingly.

3) In FL, each edge server recovers recall to almost the same level of non-distributed IDS in all attack classes; This is demonstrated by the feature distance from non-distributed IDS, which increases for independent learning and decreases for FL in all attack classes. Our results prove the effectiveness of FL over independent learning [12] and ensure that FL can attain detection rates close to that of the non-distributed IDS [13] in the context of XAI.

The remainder of this letter is organized as follows. Section II outlines our system. Section III describes the experimental conditions, Section IV presents the results and discussion, and Section V concludes this letter.

II. System Model

A. System Overview

Figure 1 shows the overview of a distributed IDS. Edge servers in the distributed IDS are placed in mobile networks (i.e., at the edge of a wide-area network). Each edge server first generates a locally-trained model (hereafter, local model) by learning with local data gathered form the communication traffic going through itself. In FL, every edge server uploads the model to the central server, and the central server integrates all of the local models, generates the global model, and returns it to every edge server. By repeating this operation, each edge server can classify the attack patterns learned by others.

Figure 2 illustrates the process of attack detection in each edge server, which consists of two phases: learning and detection. In the learning phase, each edge server collects the traffic flows relayed through itself, performs preprocessing such as normalization, and splits the flows into training and validation data. Each edge server then generates a local model by learning with the data and, in FL, sends the local model to the central server and receives the global model. In the detection phase, each edge server monitors the traffic flows relayed through itself and detects attack traffic by utilizing the trained model generated in the learning phase, i.e., the local model for independent learning or the global model for FL, and employs a XAI technique to evaluate feature contributions for detected attack traffic to enhance interpretability.

B. Training Local Models at Edge Servers

We employ a commonly used DL algorithm, MLP [14], at each edge server. According to Eq. (1), each edge server independently optimizes the parameters, i.e., weights and biases, of its local model by learning from local training data.

$$x_i^{(l+1)} = f\left(\sum_k w_{ki}^{(l)} x_k^{(l)} + b_i^{(l+1)}\right), \tag{1}$$

where $x_i^{(l+1)}$ is the output of neuron i at layer $l+1$, f is a nonlinear activation function, $w_{ki}^{(l)}$ is the weight attached to the link from neuron k at layer l to neuron i, $x_k^{(l)}$ is the output of neuron k at layer l, and $b_i^{(l+1)}$ is the bias for neuron i. The neurons in the first layer correspond to the features of a monitored traffic flow, and those in the last layer indicate the probabilities of the flow belonging to attack classes.

C. Creating Global Model at Central Server

After every edge server creates its own local model, the central server aggregates them into the global model. We employ the most basic form of FL, federated averaging [15]. This algorithm is more robust to unbalanced local training data and needs fewer rounds of communication than the standard algorithm, federated stochastic gradient descent, to train the global model [15], making it suitable for our distributed IDS. Federated averaging iteratively averages the parameters of the local models according to the algorithm [15], as follows.

$$G^{(t+1)} = G^{(t)} + \frac{\eta}{n}\sum_{j=1}^{c}\left(L_j^{(t+1)} - G^{(t)}\right), \tag{2}$$

where $G^{(t+1)}$ and $G^{(t)}$ are the parameter set of the global model at the $(t+1)$-th aggregation and that at the t-th aggregation, respectively, $L_j^{(t+1)}$ is the parameter set of edge server j's local model and c is the number of the edge servers selected for the $(t+1)$-th aggregation, n is the total number of edge servers, and η is a coefficient.

D. Evaluating Feature Contributions to Attack Traffic

We use a representative solution to explain ML, SHAP [7], which can attribute a model's attack classification of a flow to features of the flow. The contribution of feature i to the

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 07:59:22 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NUMBER OF FLOWS IN OUR EVALUATIONS USING UNSW-NB15 [11]

Attack class	Number of flows		
	Original	Training	Validation & Test
Normal (N)	2,218,764	150,000	332,815
Generic (G)	215,481	150,000	32,323
Fuzzer (F)	24,246	150,000	3,637
Exploit (E)	44,525	150,000	6,679
Reconnaissance (R)	13,987	150,000	2,099
DoS (D)	16,353	150,000	2,453

classification of a flow is evaluated by using feature *i*'s SHAP value, denoted by ϕ_i, which is defined as

$$\phi_i = \sum_{S \subseteq M \setminus \{i\}} \frac{|S|!(|M| - |S| - 1)!}{|M|!} (v(S \cup \{i\}) - v(S)), \quad (3)$$

where *M* is the set of all features used in a model, *S* is a subset of *M* excluding feature *i*, and *v* is a function quantifying the output of the model given a subset. We use the Kernel SHAP method [7] for this computation.

III. EVALUATION METHOD

In the evaluations, we set the number of edge servers to 100. We provide the details of the evaluations below.

A. Dataset

The UNSW-NB15 dataset, created in the Australian Center for Cyber Security, contains a mixture of real normal flows and synthetic attack flows for evaluating IDSs [11]. From the 10 classes of attacks recorded in this dataset, we selected six target classes: Normal, Generic, Fuzzer, Exploit, Reconnaissance, and DoS, each of which originally contains over 10,000 flows; this number must be large enough to evaluate our distributed IDS. Table I summarizes the numbers of flows in each attack class, and we abbreviate these attack classes as attacks N, G, F, E, R, and D. We then created a multi-class classification model with the six classes. The selected flows are divided in a ratio of 7:1.5:1.5, each of which is assigned to the training data and validation data in the learning phase and simulated monitored traffic (called test data) in the detection phase, respectively. The original ratio among the number of flows in every attack class was kept in the validation and test data. Meanwhile, to ensure that every attack class has equal weight on the trained model, we set the number of training flows for each attack class to 150,000 and duplicated the flows in an attack class with less than 150,000 flows. A total of 900,000 training flows is learned in a non-distributed IDS, whereas it is randomly divided into 100 different sets, each of which has 9,000 flows and is allocated to each edge server in a distributed IDS. This dataset consisted of 201 features (see the description of features in UNSW-NB15 [11]). Note that we did not divide the test data and allocated the total number of tested flows to each edge server in the distributed IDS as in the non-distributed IDS because we focus on the difference in the models between the distributed and non-distributed IDSs.

B. Preprocessing

The dataset is preprocessed in a form that can be learned by MLP, as follows.

1) One-Hot Encoding: When a feature is a categorical variable, we use one-hot encoding to show whether or not each flow belongs to a category.

2) Box-Cox Transformation and Outlier Removal: When a feature is a numerical variable covering a wide range, we correct the bias of the values of the feature by Box-Cox transformation and remove the flows, each of which has a feature value below the 1st percentile value or above the 99th percentile value.

3) Normalization: When a feature is a numerical variable, we also resize the values of the feature to relative numbers between [0, 1] by converting the minimum value to 0.0 and the maximum value to 1.0.

C. Training Local Models and Creating Global Model

We used Keras, an API for Tensorflow [16], to build a local model in each edge server. Our MLP was a fully-connected network of seven layers, including an input layer, five hidden layers using the rectified linear unit (ReLU) functions, and an output layer using the softmax function. Each of the layers contained 201, 1024, 528, 246, 128, 64 and 6 neurons, respectively. We applied dropout with strength 0.01. We used the cross entropy error as the loss function and the *Adam* optimizer. We set the number of epochs to 50, following preliminary experiments using validation data. In addition, we used Flower [17] to build a global model and set the number of aggregations of local models to 5.

IV. RESULTS AND DISCUSSION

We evaluate our distributed IDS in terms of recall, feature distances among attack classes, and the number of primary features. The recall of an attack class is the percentage of the number of correctly detected flows in the class over the total number of tested flows in the class. We use this metric to measure the attack detection rate because, unlike other standard metrics such as accuracy and precision, recall is not affected by the imbalance in the number of test flows among attack classes, specifically the high ratio of attack N to the others. The feature distance between two feature sets is calculated by using Euclidean norm: $\sqrt{\sum_{i \in M} (\phi_i - \psi_i)^2}$, where $\{\phi_i\}$ and $\{\psi_i\}$ are the respective SHAP values of two feature sets. The shorter the feature distance between the two feature sets, the higher the similarity between them. The primary features in an attack class are defined as the features whose cumulative SHAP values do not exceed 90% of the total in the attack class for the non-distributed IDS.

Figure 3 summarizes the recall for each attack class in the non-distributed IDS, and that with independent learning and FL in the distributed IDS. For independent learning, the recall refers to the average among all the edge servers, and the error bar indicates the range between the maximum and minimum. Figure 4 shows an example of the SHAP values of features, where we show the top 20 features in descending order of absolute SHAP values of features in attack G or F.

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 07:59:22 UTC from IEEE Xplore. Restrictions apply.

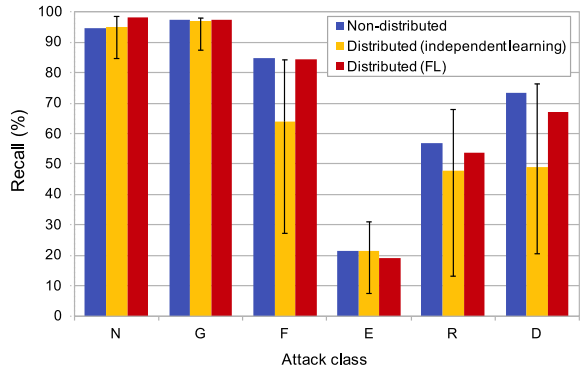


Fig. 3. Recall of all attack classes.

TABLE II
NUMBER OF PRIMARY FEATURES

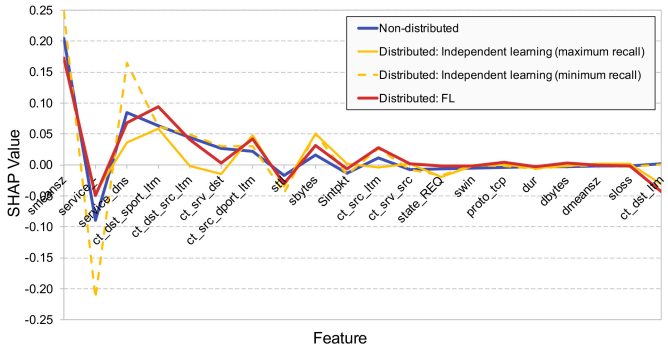
Attack class	Number of primary features
N	21
G	10
F	27
E	30
R	25
D	18

1) Non-Distributed IDS: The recall of attacks N, G, F, E, R, and D are 94%, 97%, 85%, 21%, 57%, and 74%, respectively. The detection performance varies substantially among the attack classes in UNSW-NB15; this is consistent with previous studies [2], [18], [19]. Although a few attack classes are not detected with high recall, particularly attack E, the classification of attack N from the other attacks reached 94%; this means binary classification of whether or not a flow is an attack is accurate. This is indicated by the distance from the features of an attack class (except attack N) to attack N and the average distance from the features of the attack class to another class (except attack N) as shown in Fig. 5. The former distance is between 0.48 and 0.55, while the latter distance is much smaller, between 0.21 and 0.32.

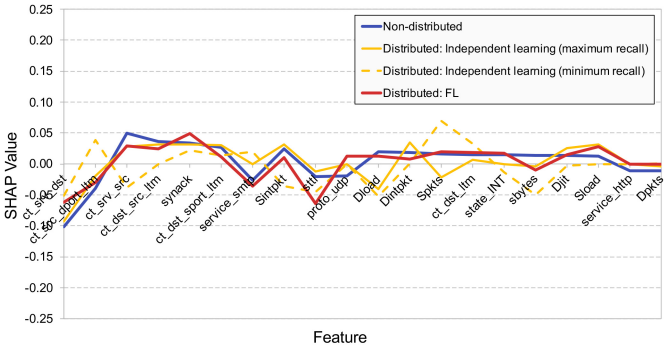
Moreover, among the attack classes excluding attack N, an attack class of higher recall (shown in Fig. 3) exhibits a larger distance from its features to the features of another attack class (shown in Fig. 5). IDS can classify an attack class whose features are far from those of the other attacks relatively easily. Attack G demonstrates the highest recall, 97%, which corresponds to the largest feature distance to the other attack classes, 0.32. In contrast, attack E, which has the lowest recall, 21%, shows the smallest feature distance to the other attack classes, 0.21, which leads to many false detections.

For the sake of clarity, we also explain the results of accuracy for each attack class (the percentage of the number of correctly classified flows with regard to the class over the total number of tested flows). The accuracies of attacks N, G, F, E, R, and D are 95%, 100%, 98%, 95%, 100%, and 99%, respectively, but these high accuracies arise from the high ratio of attack N in the test flows. We believe this metric will not contribute to analyzing the detection performance associated with feature distances.

2) Independent Learning in Distributed IDS: The recall in the non-distributed IDS degrades differently depending on the attack class as shown in Fig. 3. The recall of attack G decreases by only 0.3% due to being distributed, and the difference in recall among edge servers is relatively small



(a) Attack G



(b) Attack F

Fig. 4. Top 20 SHAP values in attack G and attack F.

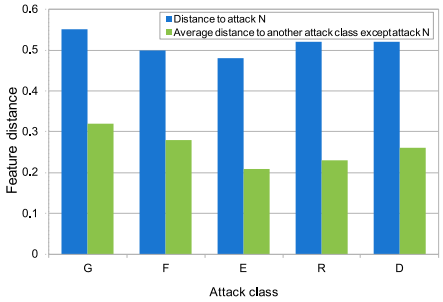


Fig. 5. Feature distances from attack classes excluding attack N in non-distributed IDS.

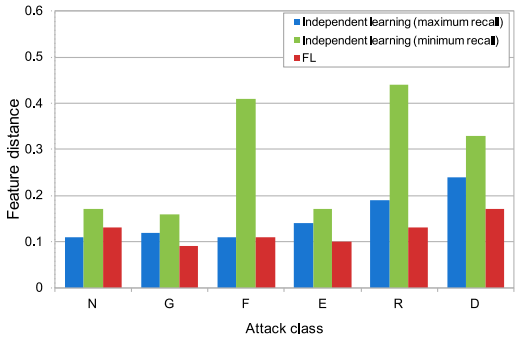


Fig. 6. Feature distances of corresponding attack classes between non-distributed IDS and distributed IDS.

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 07:59:22 UTC from IEEE Xplore. Restrictions apply.

(11%). In contrast, the average recall of attacks F and D drop considerably by 21% and 25%, respectively. Moreover, in attacks F, R, and D, there are large differences in recall among the edge servers (57%, 55% and 56%, respectively). The difference in the recall decreases between attack G and attacks F, R, and D coincides with the maximum feature distances of corresponding attack classes between the non-distributed IDS and the distributed IDS, as shown in the case of independent learning (minimum recall) in Fig. 6.

We conclude that the above difference must relate to the number of primary features in each attack class presented in Table II. Attack G only has 10 primary features as shown in Fig. 4(a), which minimizes the effect of being distributed. In contrast, the numbers of primary features in attacks F, R, and D are 27, 25, and 18, which are considerably larger than that of attack G. Figure 4(b) shows the top 20 SHAP values in attack F, which indicates no prominent features and more than 20 features required to classify attack F. As the number of primary features increases in an attack class, the primary features of the class are more likely to overlap with those of another attack class; this results in lower recall in the class. Due to the large number of primary features, the reduction in training data for being distributed has a stronger effect.

Note that attack N, which has many primary features (21), exhibits little decrease in recall. This may be because the feature distance between attack N and the other attacks is sufficiently large as previously explained. Also note that the recall of attack E does not decrease and its variance is relatively small, though this attack has the most primary features (30). This may be because its recall in the non-distributed IDS is too low to degrade in the distributed IDS.

3) Federated Learning in Distributed IDS: Applying FL to the distributed IDS leads to recovery in recall, as shown in Fig. 3. In attacks G, F, and E, the recall with FL are equivalent to those in non-distributed IDS. This is demonstrated by the relatively small feature distances from the case of non-distributed IDS (0.09, 0.11, and 0.10, respectively) as shown in Fig. 6. Attack D exhibits the least recovery from non-distributed IDS, and attack R is the second least. This corresponds to the feature distance from the case of non-distributed IDS: attack D exhibits the longest distance, 0.17, and that of attack R is the second longest, 0.13. Note that attack N has the same feature distance as attack R (0.13), but this resulted from the improvement in recall from the non-distributed IDS. Overall, FL can mitigate the degradation in detection rate resulting from the reduction in training data in each edge server of the distributed IDS. This is ensured by the feature distance from the non-distributed IDS; FL can reduce the increased distance for independent learning in all attack classes.

V. Conclusion

We applied FL to a distributed IDS and evaluated the extent to which FL could recover recall in terms of feature distance calculated using SHAP values. Through computer simulations, we revealed that FL can recover recall to almost the same ratio as non-distributed IDS for all attack classes, and the improvement from independent learning was at most 21.0%. Although the feature distance from non-distributed IDS can be large for independent learning, especially in attack classes with more primary features, it is shortened with the use of FL. In the future, as traffic passing through edge servers changes over time, we will dynamically adjust the training data for each FL round and evaluate FL through XAI.

References

[1] N. Chaabouni, M. Mosbah, A. Zemmari, C. Sauvignac, and P. Faruki, "Network intrusion detection for IoT security based on learning techniques," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2671–2701, 3rd Quart., 2019.

[2] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.

[3] L. Lavaur, M.-O. Pahl, Y. Busnel, and F. Autrel, "The evolution of federated learning-based intrusion detection and mitigation: A survey," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 3, pp. 2309–2332, Sep. 2022.

[4] M. H. Bhavsar, Y. B. Bekele, K. Roy, J. C. Kelly, and D. Limbrick, "FL-IDS: Federated learning-based intrusion detection system using edge devices for transportation IoT," *IEEE Access*, vol. 12, pp. 52215–52226, 2024.

[5] Y. Fan, Y. Li, M. Zhan, H. Cui, and Y. Zhang, "IoTDefender: A federated transfer learning intrusion detection framework for 5G IoT," in *Proc. IEEE 14th Int. Conf. BigDataSE*, 2020, pp. 88–95.

[6] N. Moustafa, N. Koroniotis, M. Keshk, A. Y. Zomaya, and Z. Tari, "Explainable intrusion detection for cyber defences in the Internet of Things: Opportunities and solutions," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 3, pp. 1775–1807, 3rd Quart., 2023.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. NIPS*, 2017, pp. 4768–4777.

[8] M. Wang, K. Zheng, Y. Yang, and X. Wang, "An explainable machine learning framework for intrusion detection systems," *IEEE Access*, vol. 8, pp. 73127–73141, 2020.

[9] M. Sarhan, S. Layeghy, and M. Portmann, "Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection," *Big Data Res.*, vol. 30, Nov. 2022, Art. no. 100359.

[10] P. Barnard, N. Marchetti, and L. A. DaSilva, "Robust network intrusion detection through explainable artificial intelligence (XAI)," *IEEE Netw. Lett.*, vol. 4, no. 3, pp. 167–171, Sep. 2022.

[11] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. MilCIS*, 2015, pp. 1–6.

[12] S. Abdulrahman, H. Tout, H. Ould-Slimane, A. Mourad, C. Talhi, and M. Guizani, "A survey on federated learning: The journey from Centralized to distributed on-site learning and beyond," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5476–5497, Apr. 2021.

[13] M. A. Ferrag, O. Friha, L. Maglaras, H. Janicke, and L. Shu, "Federated deep learning for cyber security in the Internet of Things: Concepts, applications, and experimental analysis," *IEEE Access*, vol. 9, pp. 138509–138542, 2021.

[14] J. Patterson and A. Gibson, *Deep Learning: A Practitioner's Approach*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[15] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[16] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX OSDI*, 2016, pp. 265–283.

[17] D. J. Beutel et al., "Flower: A friendly federated learning research framework," 2022, *arXiv:2007.14390*.

[18] N. Moustafa, J. Slay, and G. Creech, "Novel geometric area analysis technique for anomaly detection using trapezoidal area estimation on large-scale networks," *IEEE Trans. Big Data*, vol. 5, no. 4, pp. 481–494, Dec. 2019.

[19] P. Singh, A. Kaur, G. S. Aujla, R. S. Batth, and S. Kanhere, "DaaS: Dew computing as a service for intelligent intrusion detection in edge-of-things ecosystem," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12569–12577, Aug. 2021.

Authorized licensed use limited to: National Institute of Technology Patna. Downloaded on March 14,2026 at 07:59:22 UTC from IEEE Xplore. Restrictions apply.